

Manual operativo de datos — Sistema de pagos del Salón Anselmo

1. Base de datos: Airtable “Gestión de Pagos de Eventos”

La base funciona como memoria y registro del sistema, con 3 tablas vinculadas.

Tabla Eventos

Campo	Tipo	Propósito
Cliente / Nombre del evento	Texto	Identifica el evento (ej. “Boda García-López”)
Email	Texto	Clave usada para matchear el mail entrante con el evento
Fecha	Fecha	Fecha del evento
Monto total contratado	Currency	Monto acordado con el cliente
Estado del evento	Selección única	Confirmada / Señada / Pagada / Cancelada — estado de negocio
Estado del procesamiento	Selección única	Pendiente → Procesado por IA → Aprobado por Humano → Enviado / Rechazado — estado técnico exigido por la consigna
Total pagado	Rollup (SUM sobre Pagos.Monto)	Se calcula solo, suma todos los pagos vinculados
Saldo pendiente	Fórmula: <code>Monto total contratado - Total pagado</code>	Se calcula solo
Pagos	Link → tabla Pagos	Relación uno a muchos

Tabla Pagos

Campo	Tipo	Propósito
Evento	Link → tabla Eventos	Vincula el pago a su evento
Monto	Currency	Monto detectado por la IA en el comprobante
Fecha de pago	Fecha	Fecha detectada por la IA
Método detectado por IA	Selección única	Efectivo / Transferencia / Tarjeta / No identificado
Comprobante	Adjunto	Guarda la imagen original enviada por el cliente (trazabilidad)

Tabla Errores

Campo	Tipo	Propósito
Fecha	Fecha y hora	Momento del error
Nodo	Texto	Nombre del nodo de n8n que falló
Motivo	Texto largo	Mensaje de error capturado
ID de ejecución	Texto	ID de la ejecución de n8n, para poder auditarla

Relaciones: Eventos 1—N Pagos (un evento puede tener múltiples pagos parciales). La tabla Errores es independiente, no se relaciona con las otras dos — es un log plano.

2. Esquemas JSON de las integraciones

a) Salida del Gmail Trigger (input del flujo)

```
{
  "id": "1a02af24cab3f67e",
  "subject": "Pago cuota salon",
  "from": {
    "value": [{ "address": "cliente@email.com", "name": "Nombre Cliente" }]
  },
  "date": "2026-08-22T19:28:31.000Z"
}
```

El binario del adjunto viaja aparte, en `binary.attachment_0` (fileName, mimeType, data en base64).

b) Salida de la IA (Basic LLM Chain + Structured Output Parser)

```
{
  "output": {
    "es_comprobante": true,
    "motivo": "",
    "monto": 30000,
    "fecha": "2026-08-20",
    "metodo": "Transferencia"
  }
}
```

Este objeto es el que consume el nodo IF de validación y, más adelante, los nodos que arman los mensajes de Telegram y los registros de Airtable.

c) Respuesta del nodo Telegram “Send and Wait for Response”

```
{
  "data": {
    "approved": true
  }
}
```

Es el campo que decide, en cada uno de los 2 puntos de HITL, si el flujo continúa creando registros o se detiene.

d) Payload enviado a Airtable al crear un pago

```
{
  "fields": {
    "Evento": ["recXXXXXXXXXXXX"],
    "Monto": 30000,
    "Fecha de pago": "2026-08-20",
    "Método detectado por IA": "Transferencia"
  }
}
```

e) Payload enviado a Airtable al registrar un error

```
{
  "fields": {
    "Fecha": "2026-08-22T19:30:00.000Z",
    "Nodo": "Basic LLM Chain",
    "Motivo": "Your credit balance is too low to access the API.",
    "ID de ejecución": "48213"
  }
}
```

Principio de diseño: en cada nodo, los valores dinámicos (email, monto, fecha, IDs de registro) se toman siempre por expresión desde el nodo de origen correspondiente — nunca se hardcodean, cumpliendo el requisito de arquitectura de la consigna.